

BuildWatch — документация проекта

Версия: 29 сентября 2026 года. Прототип мониторинга строительных площадок: датированные снимки камер, календарный план, детекция техники и проверяемые сигналы инспектору.

Команда: Басков Константин — разработчик; Манаков Дмитрий — продуктовый менеджер.

Публичный прототип: <https://buildwatch.baski.pro/>.

Репозиторий: <https://github.com/BaskovKonstantin/BuildWatch>.

1. Назначение и границы

BuildWatch связывает объект, датированный снимок, распознанную технику и действующий на дату снимка этап работ. Правила выдают сигнал, если обязательная техника не обнаружена, наблюдаемый класс не соответствует этапу, техника попала в опасную зону кадра или по серии снимков не подтверждена активность. Карточка объекта дополнительно считает прогноз по графику, динамику за 30 дней и долю вердиктов инспектора.

В интерфейсе: портфель объектов (карта и список), карточка площадки, календарный план и хронология, снимки с рамками, зоны кадра, предупреждения, ИИ-помощник и отчёт. Сигнал и прогноз требуют проверки человеком: один ракурс не доказывает отсутствие техники на всей площадке.

Справочник ЛТЦ задаёт виды работ без дат. Календарь заполняется вручную или импортом CSV/XLSX с колонками «Этап», «Дата начала», «Дата окончания» (сначала предпросмотр и ошибки, запись только после подтверждения).

2. Инструкция по использованию

2.1. Главный экран (портфель)

Адрес локально: <http://127.0.0.1:8700>, публично: <https://buildwatch.baski.pro/>.

1. Откройте карту Москвы (режим по умолчанию) или переключитесь на «Список».
2. Цвет маркера: красный — есть открытые проблемы, янтарный — нужна проверка, зелёный — открытых проблем нет.
3. Нажмите маркер: справа/внизу появится краткая карточка (адрес, этап, прогресс, число сигналов, прогноз, дата последнего снимка).
4. «Открыть объект» ведёт в подробную карточку.
5. Поиск и фильтры «Все / С проблемами / Без проблем» работают и для карты, и для списка.
6. Кнопка «? Гайд» подсвечивает блоки экрана с пояснениями.
7. «Спросить BuildWatch» открывает ИИ-помощника по портфелю.

2.2. Карточка объекта

Маршрут: </objects/{id}>.

1. Выберите снимок в ленте: на кадре отображаются рамки детекции и список классов с уверенностью.
2. Блок «План и факт» показывает этап на дату снимка, ожидаемую технику и наблюдение.
3. Строка активности по серии: «предположительно работает», «вероятный простой» или «данных мало».
4. Прогноз по графику — эвристика по видимой части площадки, не процент готовности.

5. Откройте предупреждение: видны код правила (R-xx), текст «почему» и действия подтвердить / отклонить.
6. Кнопка «Зоны» на снимке: нарисуйте прямоугольник (**work / danger / storage**). Техника в **danger** даёт R-09; кран в **storage** на монтаже — R-10.
7. Загрузка нового снимка: выберите файл и дату съёмки. При наличии весов worker обрабатывает кадр; статус снимка меняется **new** → **processing** → **detected / empty / failed**.
8. Вердикт по рамке (**correct / wrong**) влияет на правила и экран качества распознавания.

2.3. Хронология и план

Маршрут: **/objects/{id}/timeline**.

1. Режимы «лента» и «календарь» событий и этапов.
2. Редактор этапов: добавление, правка дат и вида работ.
3. Импорт плана: CSV или XLSX до 2 МБ, столбцы «Этап», «Дата начала», «Дата окончания» (форматы ДД.ММ.ГГГГ или ГГГГ-ММ-ДД). Сначала предпросмотр; «Сохранить план» записывает этапы.
4. Справочник ЛТЦ без дат импортировать как календарь нельзя — система отклонит файл.

2.4. Отчёт

Маршрут: **/objects/{id}/report**.

Сводка по объекту: прогноз, динамика, качество распознавания (вердикты инспектора), план, статусы сигналов, ссылки на снимки. «Печать / PDF» — через печать браузера.

2.5. ИИ-помощник

Кнопка «Спросить BuildWatch» на главной и в карточке.

1. Задайте вопрос по данным объекта или портфеля (этап, сигналы, снимки).
2. Помощник отвечает по контексту приложения и показывает ссылки на использованные сущности.
3. Предложение сдвинуть даты этапа применяется только после отдельного подтверждения в UI.
4. Без серверного ключа Zen помощник недоступен; остальной функционал работает.

2.6. Типовой рабочий день инспектора

1. На карте найти красные/янтарные объекты.
2. Открыть объект → последний снимок → рамки и этап.
3. Разобрать открытые R-01 / R-03 (и зоны, если нарисованы).
4. Подтвердить или отклонить сигнал; при ошибке детекции пометить рамку **wrong**.
5. При необходимости обновить план или загрузить новый кадр.
6. Сформировать отчёт для передачи дальше.

3. Архитектура и структура кода

Пользователь/камера → снимок + дата → FastAPI → SQLite/PostgreSQL

↓

очередь detection_jobs

↓

worker → YOLO + SAHI → detections

↓

дата снимка + stages → rules → warnings

↓

forecast (при чтении карточки)

↓

Next.js: карта, карточка, отчёт

Путь · Назначение

[backend/app.py](#) · HTTP API FastAPI: объекты, снимки, зоны, план, сигналы, помощник

[backend/db.py](#) · SQLite/PostgreSQL, схема, CRUD

[backend/detection_service.py](#) · контракт JSON детектора, запись рамок, пересчёт правил

[backend/rules.py](#) · нормализация классов, allowed/required по этапу, R-01...R-10

[backend/forecast.py](#) · прогноз, активность серии, потребность техники, динамика 30 дней

[backend/plan_import.py](#) · предпросмотр и валидация CSV/XLSX

[backend/assistant_service.py](#) · вызов OpenCode Zen (GLM), предложения по плану

[backend/evaluate.py](#) · пересчёт warnings объекта после детекции или вердикта

[backend/detector_config.py](#) · имя модели, пути к весам, canonical_model

[scripts/worker.py](#) · опрос [detection_jobs](#), запуск инференса, retry зависших

[scripts/detect_equipment_single.py](#) · SAHI + Ultralytics YOLO, JSON на выходе

[frontend/app/](#) · страницы Next.js: [/](#), [/objects/\[id\]](#), timeline, report

[frontend/components/](#) · карта, карточка, гайд, помощник, редактор этапов

[migrations/001_production.sql](#) · схема PostgreSQL

[tests/](#) · unit-тесты правил, импорта плана, API, помощника

[docker-compose.yml](#), [Dockerfile*](#) · API, worker, frontend

3.1. Поток обработки снимка

1. [POST /api/objects/{id}/upload](#) сохраняет файл и запись [snapshots](#) со статусом [new](#), создаёт [detection_jobs](#).
2. Worker берёт задание со статусом [queued](#) / повторно ставит зависшие [processing](#).
3. [detect_equipment_single.py](#) читает JSON [{image, weights, out, model}](#), пишет JSON с [detections](#).
4. [detection_service](#) проверяет модель, [score](#) ∈ [0,1], [box](#) с положительной площадью; нормализует [label](#).
5. Старые детекции снимка удаляются; пишутся новые; статус снимка [detected](#) или [empty](#); [job](#) → [done](#).
6. [evaluate_object](#) пересчитывает [warnings](#) по правилам и зонам.
7. При ошибке инференса [job](#) → [failed](#), отсутствие техники из этого не выводится.

3.2. Инференс (SAHI + YOLO)

Параметры в [scripts/detect_equipment_single.py](#):

- модель: YOLOv8m, вход [1280](#) px, порог уверенности сохранения [0,15](#);
- нарезка SAHI: тайлы [1280×1280](#), [overlap 15%](#);
- дополнительно полный кадр ([perform_standard_pred=True](#));
- слияние дублей: GREEDYNMM, метрика IOS, порог [0,5](#), классы не смешиваются;
- устройство по умолчанию CPU; GPU допустим при наличии.

Весы: [weights/equipment_v12ft_2_best.pt](#) (не в публичном Git). Env [BUILDWATCH_DETECTOR_MODEL](#) (по умолчанию [equipment](#)).

3.3. Правила сопоставления

Этап снимка: `captured_at` внутри `date_from–date_to`, иначе резервно этап со статусом `current`.

Разрешённые классы — `STAGE_ALLOWED` по `kind` этапа (`ground`, `excavation`, `frame`, `facade`, `roof`).

Обязательные группы — `STAGE_REQUIRED` / уточнения `STAGE_REQUIRED_BY_NAME` по точному имени этапа из каталога (например котлован: экскаватор + самосвал).

Код · Условие · Результат

R-01 · Класс не в `allowed` для этапа · Сигнал о несоответствии

R-02 · `score < 0,5` или конфликт классов на рамке · Ручная проверка

R-03 · Ни один класс из обязательной группы не найден (порог правил `0,35`) · Риск снижения темпа

R-08 · На 2–3 последних снимках боксы класса почти совпадают (`IoU > 0,55`) · Вопрос об активности

R-09 · Центр бокса в полигоне `danger` · Техника в опасной зоне

R-10 · Крановый класс в `storage` на этапе монтажа (`frame` / `roof` / имя с «монтаж») · Вопрос инспектору

Вердикт рамки `wrong` исключает её из R-08...R-10 и из сравнения потребности. Категория «нет техники» не считается по кадрам в статусе `new`, `processing` или `failed`.

3.4. Прогноз и динамика

Считает `forecast_schedule` при чтении карточки (не пишется в `warnings`):

- берутся до трёх последних снимков со статусом `detected` или `empty`;
- факт этапа — плановый этап на дату снимка;
- если снимки «застряли» на предыдущем этапе, дни считаются от `date_to` или как доля прошедших дней этапа;
- каждые два открытых сигнала R-01/R-03 добавляют один день отставания;
- меньше двух снимков или нет этапа на сегодня → «недостаточно данных»;
- в тексте всегда дисклеймер: оценка по видимой части площадки.

Потребность техники: требования текущего этапа против классов на последних трёх снимках выше `0,35`.

Динамика за 30 дней: недельные точки и одна фраза (`GET /api/objects/{id}/dynamics`).

3.5. Frontend

Next.js 15.5 (App Router). Основные маршруты: `/` — портфель; `/objects/[id]` — карточка; `/objects/[id]/timeline` — хронология и план; `/objects/[id]/report` — отчёт. Карта — Leaflet + OSM. Состояние фильтров на главной хранится в URL. PageGuide якорит подсказки к DOM-блокам. Переходы между страницами — View Transitions (`view-morph`) с запасным `spring-slide`. Тема Editorial: светлая/тёмная по системе.

3.6. ИИ-помощник (реализация)

`assistant_service.py` вызывает OpenCode Zen (`BUILDWATCH_ZEN_URL` / go-URL), модель по умолчанию `glm-5.3-flash`. Ключ: `BUILDWATCH_ZEN_KEY` или сохранённый ключ OpenCode.

`BUILDWATCH_ZEN_REASONING_EFFORT`: `low` (по умолчанию), `medium`, `high`, `xhigh`, `max`, `none`, `adaptive`.

Предложения смены дат — отдельные сущности с `confirm`-эндпойнтом; без `confirm` план не меняется.

3.7. HTTP API (основное)

Метод · Путь · Назначение

GET · `/api/health` · сервис и доступность детектора

GET/POST · `/api/objects` · список / создание объекта

GET · `/api/objects/{id}` · карточка со снимками, `warnings`, прогнозом

POST · `/api/objects/{id}/upload` · загрузка снимка

POST · `/api/snapshots/{id}/detect` · постановка в очередь детекции

GET · [/api/snapshots/{id}](#) · снимок и детекции
POST · [/api/warnings/{id}](#) · статус сигнала
POST · [/api/detections/{id}/verdict](#) · вердикт по рамке
POST · [/api/objects/{id}/plan/preview](#) · предпросмотр CSV/XLSX
PUT · [/api/objects/{id}/stages](#) · запись этапов
GET/POST/DELETE · [/api/objects/{id}/zones](#) · зоны кадра
GET · [/api/objects/{id}/dynamics](#) · динамика 30 дней
POST · [/api/assistant/query](#) · вопрос помощнику
POST · [/api/assistant/proposals/{id}/confirm](#) · подтвердить сдвиг плана

Переменные окружения: [BUILDWATCH_SQLITE_PATH](#), [BUILDWATCH_DATABASE_URL](#),
[BUILDWATCH_DETECTOR_MODEL](#), [BUILDWATCH_AUTH_REQUIRED](#), [BUILDWATCH_AUTH_SECRET](#),
[BUILDWATCH_ZEN_KEY](#), [BUILDWATCH_ZEN_MODEL](#), [BUILDWATCH_ZEN_REASONING_EFFORT](#),
[BUILDWATCH_ZEN_TEMPERATURE](#). Ключи не класть в Git и во frontend.

4. Модель данных

Таблица · Значимые поля · Назначение

[objects](#) · [id](#), [name](#), [type](#), [district](#), [address](#) · Строительные объекты
[snapshots](#) · [object_id](#), [captured_at](#), [status](#), [filename](#), [src](#) · Снимки и состояние обработки
[detections](#) · [snapshot_id](#), [model](#), [label](#), [score](#), [x1...y2](#), [verdict](#) · Классы, рамки, вердикт инспектора
[stages](#) · [object_id](#), [position](#), [kind](#), [name](#), [date_from](#), [date_to](#), [status](#) · Календарные этапы
[warnings](#) · [snapshot_id](#), [rule](#), [severity](#), [why](#), [source](#), [status](#) · Проверяемые отклонения
[site_zones](#) · [object_id](#), [name](#), [kind](#), [polygon_json](#) · Зоны кадра в долях 0–1
[detection_jobs](#) · [snapshot_id](#), [model](#), [status](#), [attempts](#), [error](#) · Очередь инференса
[catalog](#) · [code](#), [name](#), [applies](#) · Виды работ ЛТЦ
[users](#), [audit_log](#) · пользователь, роль, действие, время · Доступ и аудит

Демо — SQLite; production-схема — [migrations/001_production.sql](#). Кадры на диске, в БД — пути.

5. Данные модели и ограничения

Целевые восемь классов: самосвал, экскаватор, каток, кран-манипулятор, бетономеситель, бульдозер, грузовик, автокран. Вспомогательные классы онтологии не обещают равное качество. Источники и лицензии: [docs/ml-models.md](#), [docs/external-training-data.md](#), [docs/mocs-dataset.md](#), [docs/openimages-training-data.md](#).

Ограничения: веса и материалы организатора ([context/dgp_extract/](#)) не в публичном Git; без весов новая детекция недоступна; один ракурс не покрывает площадку; внешние метрики и MOCS/Thalos не равны оценке текущего [equipment](#) v12 на Москве; прогноз и R-08 не доказывают готовность или простой; зоны в долях кадра, не GPS; экран качества — вердикты инспектора, не precision/recall; маска обзора камеры (R-11) не реализована.

6. Сборка, установка и запуск

Windows

1. Python 3.12, Node.js 20+; `pip install -r requirements.txt`; `npm --prefix frontend ci`.
2. Для новой детекции: PyTorch, Ultralytics, файл [weights/equipment_v12ft_2_best.pt](#).
3. При наличии [context/dgp_extract/](#): `python scripts/seed_public_demo.py` (или `backend/seed.py` / `backend/demo_projects.py`).

4. `powershell -ExecutionPolicy Bypass -File scripts/start-buildwatch.ps1` — UI :8700, API health :8600/api/health.

Linux / WSL

Из корня: `cd backend && ../.venv/bin/uvicorn app:app --host 127.0.0.1 --port 8600; ../.venv/bin/python scripts/worker.py; npm --prefix frontend run dev -- --port 8700`.

Docker Compose

`docker compose up --build` — API, worker, frontend. Seed из `context/dgp_extract/`. Каталог `weights/` монтируется отдельно. `BUILDWATCH_ZEN_KEY` только в env сервера.

Проверки: `python -m unittest discover -s tests -v; npm --prefix frontend run build`; health и загрузка снимка в UI.

7. Сценарий проверки прототипа

1. Главный экран: карта, легенда, маркеры.
2. Объект с проблемой → «Открыть объект».
3. Датированный снимок и рамки; при весах — новый кадр.
4. Этап на дату, ожидаемая техника, активность по серии.
5. Прогноз и фраза за 30 дней.
6. Сигнал R-03 или R-01; при зоне — R-09.
7. Вердикт, блок качества, отчёт.

8. Установка камер

Камера должна перекрывать рабочие зоны и маршруты техники, сохранять время съёмки и идентификатор площадки. Нужны фиксированные ракурсы, разрешение для удалённой техники, учёт освещения и осадков. Подробности: руководство по установке камер (`camera-installation-recommendations.md`).